

改变，这通常需要更深层次的理解和处理能力。COUNTERFACT 能够评估模型对编辑后知识的理解和反应，进而衡量模型的**泛化性**和**局部性**。文献 [27] 在 ZsRE 和 COUNTERFACT 数据集的基础上，使用 GPT-4 生成相应问题的反向问题、推理问题和实体替换问题，构造了**可迁移性**数据集。

此外，研究者还针对不同领域和任务开发了专门的数据集，如 **Hallucination**[10] 用于纠正 GPT 语言模型中的幻觉，**ConvSent**[20] 用于评估模型在修改对话代理对特定主题的情感时的效果。表 5.1 从数据类型、样本数量以及输入输出形式方面总结了一些模型编辑相关数据集。这些数据集涵盖了从事实检查、知识关联到特定领域等多种类型，体现了模型编辑技术在不同场景中的应用潜能。

本节介绍了模型编辑的定义、性质、评估方法以及常用数据集，对模型编辑任务做出了详细的解释和说明。接下来，第 5.2 节将对模型编辑方法进行系统性概述，将其分为外部拓展法和内部修改法，并介绍每类方法的代表性工作。第 5.3 节和第 5.4 节将分别深入探讨外部拓展法中的 T-Patcher 方法和内部修改法中的 ROME 方法，通过这两种代表性方法，帮助读者更加细致地理解模型编辑方法的研究过程。最后，第 5.5 节将全面介绍模型编辑的实际应用，并分别举例说明解决思路。

5.2 模型编辑经典方法

冒险游戏中的勇者需要升级时，可以从内外两个方面进行改造。外部改造主要通过置办新的道具和装备，它们能够赋予勇者新的能力，同时保留其原有技能。内部改造则相当于去锻炼自身，通过增加智力、体力、法力等属性，从自我层面获得提升。如果将大语言模型比作冒险游戏中的勇者，那么模型编辑可被看作一种满足“升级”需求的方法，可以分别从内外两个角度来考虑。本文参考已有工作 [16, 25, 27, 28]，将现有编辑方法分为**外部拓展法**和**内部修改法**。概括来说，外部拓展

法通过设计特定的训练程序，使模型在保持原有知识的同时学习新信息。内部修改法通过调整模型内部特定层或神经元，来实现对模型输出的精确控制。图 5.4 给出了模型编辑方法的分类：外部拓展法包括知识缓存法和附加参数法，内部修改法包括元学习法和定位编辑法。接下来，本节将对每类编辑方法展开介绍。

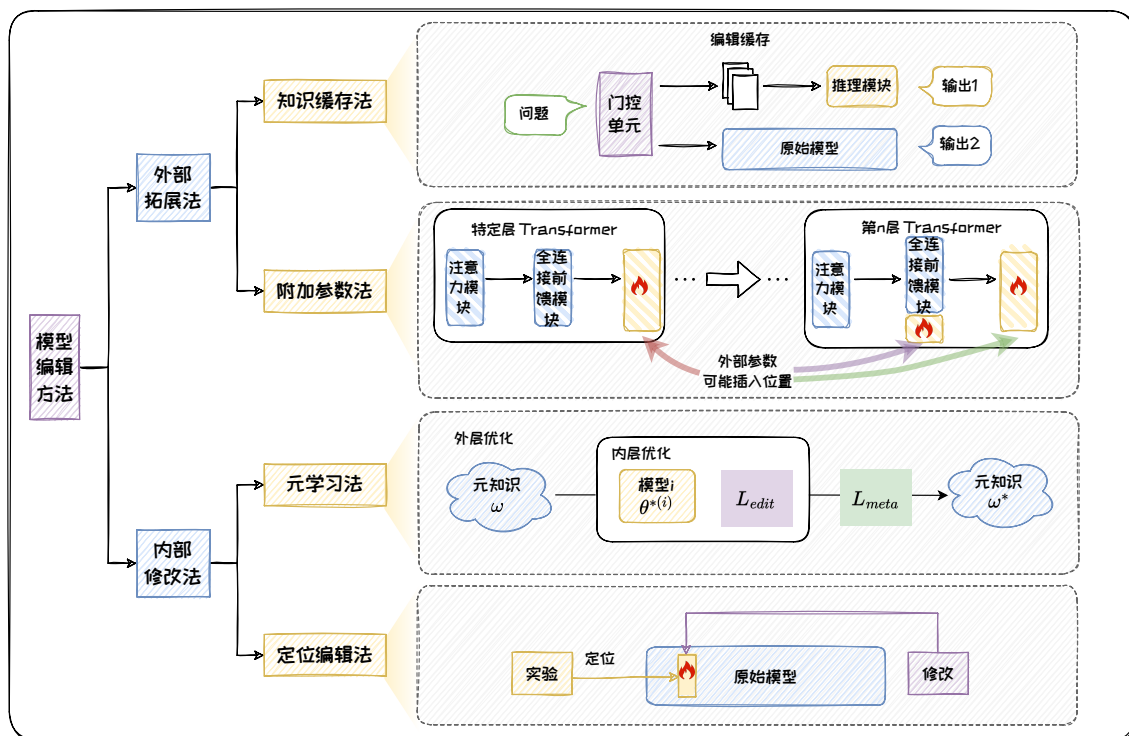


图 5.4: 模型编辑方法分类图。

5.2.1 外部拓展法

外部拓展法的核心思想是将新知识存储在附加的外部参数或外部知识库中，将其和原始模型一起作为编辑后模型。这种方法尤其适合具有良好可扩展性的预训练大语言模型，因为它提供了足够的空间来容纳大量参数，能够存储更多新知识。此外，该方法不会改变原始模型参数，可降低对模型内部预训练知识的干扰。

根据外部组件是否直接整合进模型本身的推理过程，外部拓展法又可划分为知识缓存法和附加参数法。延续冒险游戏的比喻，知识缓存类似于勇者的技能书，

需要时可以查阅获取特定知识；而附加参数则如同可升级的装备，直接增强勇者
的整体能力。

1. 知识缓存法

知识缓存法中包括三个主要组件，分别为门控单元、编辑缓存和推理模块。**编辑缓存**充当一个知识存储库，用于保存需要修改的知识，这些知识由用户通过不同的形式指定。**门控单元**用于判断输入问题与编辑缓存中的知识的相关程度，可通过分类 [20] 或噪声对比估计 [21] 等任务进行训练。**推理模块**获取原始输入问题和编辑缓存中的知识作为输入，通过监督训练的方式学习预测用户期望的结果。

在推理时，门控单元首先判断输入的问题是否与编辑缓存中的某个知识点相关，如果相关，则从编辑缓存中取出该知识点，将其与输入一并交给推理模块，由推理模块给出答案；如果不相关，则用原始模型去推理给出答案。图 5.5 为知识缓存法示意图，其中，假设编辑缓存中存储的是与斑马肤色有关的知识，则问题“鸵鸟会飞吗？”由门控单元判断为与编辑缓存中的所有知识点都不相关的问题，因此由原始模型推理出答案；问题“皮肤为黑色的马是什么马？”由门控单元判断为与“斑马的肤色”这个知识点相关的问题，因此由训练好的推理模块给出修改后答案。

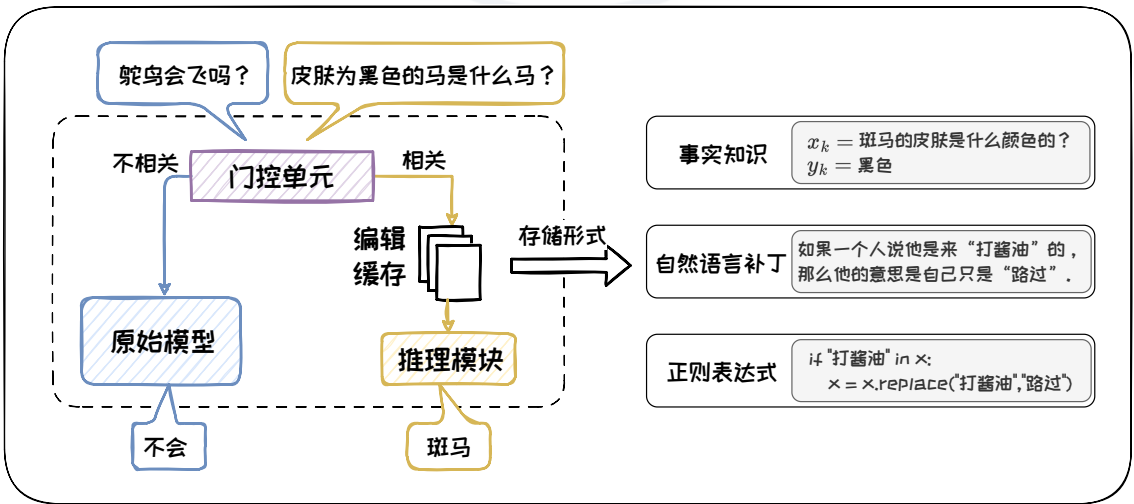


图 5.5: 知识缓存法示意图。

此外,编辑缓存中知识点的存储形式可以分为事实知识、自然语言补丁和正则表达式三种。**事实知识**以问题-答案对 (x_k, y_k) 存储编辑实例,这种存储形式适用于答案明确的事实性问题,**SERAC**[20] 是一种代表性方法。**自然语言补丁 Language Patch**[21] 按照“如果……那么……”的句式描述编辑知识,类似于 Prompt。这种存储形式适用于修正模型对自然语言中非字面含义语句的理解,且便于人们创建、编辑或删除补丁,使得模型能够通过人类反馈不断修正输出。**正则表达式**是一种基于文本匹配和替换的技术,它使用特定的模式来识别和修改文本中的特定部分,适用于精确的文本语义替换。这是一种早期的原始方法,然而由于编写复杂、泛化性低,因此在模型编辑中并不常用。

知识缓存法直接通过编辑缓存中的信息进行检索,不依赖目标标签的梯度信息,因此可以简化模型编辑过程,使其更加高效直接。然而,这种从外界获取知识的方式相当于在让大语言模型进行求助,而并非将新的知识真正内化为自己的一部分。附加参数法对这种局限进行了改良。

2. 附加参数法

与知识缓存法相比,附加参数法可以将外部参数整合进模型结构,从而有效利用和扩展模型的功能。这类方法的思想与参数高效微调中的**参数附加方法**类似,都是将外部参数插入到模型中的特定位置,冻结原始模型,只训练新引入的参数以修正模型输出,如图 5.6。具体而言,不同方法将外部参数插入到模型的不同位置。例如,**CALINET**[6] 和 **T-Patcher**[13] 通过修改模型最后一层 Transformer 的**全连接前馈模块**来实现。**CALINET** 首先通过一种对比知识评估方法找出原始模型的知识错误,然后在模型最后一个全连接前馈模块添加一个新的参数矩阵,并通过最小化校准数据上的损失来训练新参数,以纠正模型的错误。**T-Patcher** 与此类似,同样是在原始模型的最后一个全连接前馈模块引入有限数量的可训练神经元,每个神经元对应一个知识点。关于 **T-Patcher** 的具体介绍将在第 5.3 节给出。

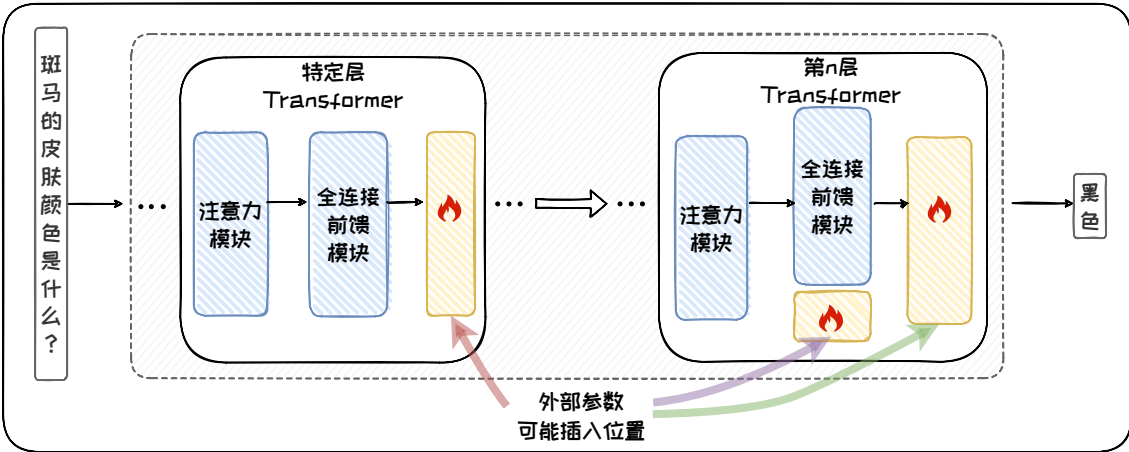


图 5.6: 附加参数法示意图。

GRACE[10] 则将外部参数以适配器的形式插入模型的特定 Transformer 层中，插入位置随模型变化而变化，如 BERT 的倒数第 2 层和 GPT2-XL 的第 36 层。其中，适配器是一个用于缓存错误知识（Keys）和对应的修正值（Values）的键值存储体，被称作“codebook”。在 codebook 中，每个错误知识都有一个对应的修正值，以及一个用于匹配相似输入的延迟半径，且随着时间的推移，codebook 会被持续更新。延迟半径用于判断当前输入是否与 codebook 中的任何错误相似，如果是，则应用相应的修正值进行编辑。

综上，知识缓存法通过引入编辑缓存机制，有效地辅助模型在庞大的知识体系中迅速定位并检索最新信息；附加参数法通过引入额外参数，实现了对模型特定输出的精细调整。这两种方法的核心优势在于**对原始模型的最小化干预**，保证了模型编辑的**局部性**。

然而，外部拓展法的实际有效性在很大程度上取决于对知识的存储与检索能力，这种依赖会导致存储资源需求的增加。因此，在具体应用时，我们需要在**保证模型局部性和应对存储限制**之间寻求平衡。

5.2.2 内部修改法

与需要额外存储空间的外部拓展法不同，内部修改法能够让模型在不增加物理存储负担的情况下直接优化自身。**内部修改法旨在通过更新原始模型的内部参数来为模型注入新知识**，能够优化模型的自我学习和适应能力，提高其在特定任务上的表现，而不是仅仅停留在表面的知识积累。内部修改法又可以分为**元学习**和**定位编辑法**。其中，元学习法通过“学习如何学习”来获取元知识，再基于元知识实现模型编辑；定位编辑法则专注于对模型局部参数的修改，首先识别与目标知识最相关的模型参数，然后仅更新这些特定参数，通过“先定位后编辑”的策略节省更新模型所需成本。

1. 元学习法

元学习指的是模型“学习如何学习”（Learning to Learn）的过程。基于元学习的模型编辑方法，旨在让模型“学习如何编辑”（Learning to Edit），核心思想是使模型从一系列编辑任务中提取通用的知识，并将其应用于未见过的编辑任务，这部分知识被称为**元知识** ω [12]。元知识是模型在进行编辑前可以利用的知识，包括优化器参数 [24]、超网络 [2, 19] 等多种形式。元知识的训练过程被称为**元训练**，其目标是获得一个较好的元知识 ω ，使得后续的每次编辑只需少量样本即可快速收敛。

元训练过程可以看作一个双层优化问题 [12]，如公式 5.5 所示。双层优化（Bilevel Optimization）是一个层次化的优化框架，其中，内层优化问题可作为外层优化问题的约束。基于元学习的编辑方法如图 5.7 所示。图中，内层优化是模型在不同编辑任务上的优化，外层优化是元知识在验证集中的编辑任务上的综合优化。

$$\begin{aligned} \omega^* &= \arg \min_{\omega} \sum_{i=1}^n L_{\text{meta}}(\theta^{*(i)}(\omega), \omega, D_k^{\text{val}(i)}) \\ \text{s.t. } \theta^{*(i)}(\omega) &= \arg \min_{\theta} L_{\text{edit}}(\theta^{(i)}, \omega, D_k^{\text{train}(i)}). \end{aligned} \quad (5.5)$$

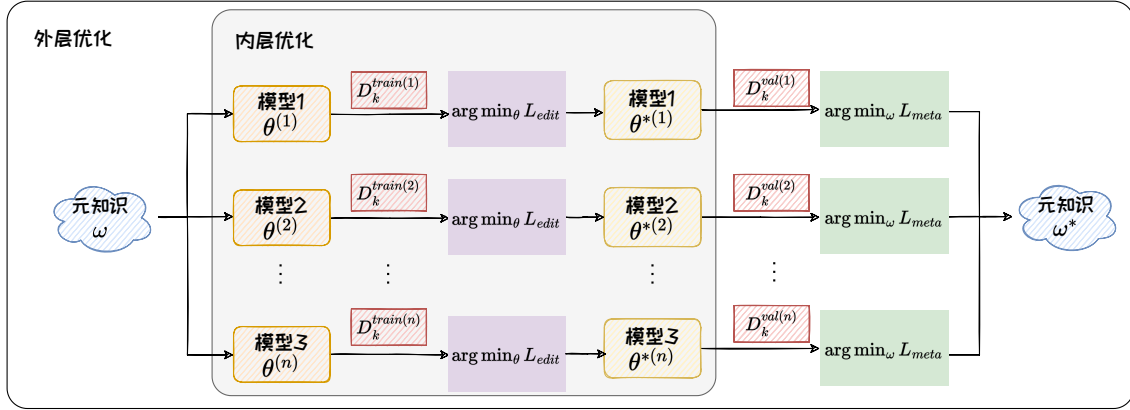


图 5.7: 元学习法示意图。

内层优化问题旨在学习模型在具体编辑任务上的参数。在内层中，设共有 n 个编辑任务，则对于第 i 个编辑 ($i \in [1, n]$ 且 $i \in \mathbb{Z}^+$)，用 $D_k^{(i)}$ 表示该次编辑涉及到的有关知识点 k 的数据集，其中 $D_k^{(i)}$ 是由 (x_k, y_k) 组成的问题-答案对集合，可划分为训练集 $D_k^{\text{train}(i)}$ 和验证集 $D_k^{\text{val}(i)}$ 。设模型原始参数为 $\theta^{(i)}$ ，在 $D_k^{(i)}$ 上优化后的参数为 $\theta^{*(i)}$ ，则内层优化是在元知识 ω 的基础上更新对应编辑任务上的模型参数的过程。在内层优化中， L_{edit} 是有关每次编辑的损失函数，其含义为希望能够最小化编辑后模型 $\theta^{*(i)}$ 在 $D_k^{\text{train}(i)}$ 上的预测误差，如分类任务中的交叉熵函数。

外层优化问题旨在学习可以泛化到其他编辑任务上的元知识。外层优化通常在验证集 $D_k^{\text{val}(i)}$ 上进行，希望能够根据模型 $\theta^{*(i)}$ 在其对应编辑验证集上的损失来更新元知识 ω 。外层优化中的 L_{meta} 是有关元学习的损失函数，其含义为希望能够找到一个元知识 ω ，使得在该知识的基础上进行优化后的模型在所有 $D_k^{\text{val}(i)}$ 上的预测误差之和最小。

ENN[24] 将元知识看作优化器参数，通过更新优化器参数，使后续编辑中模型参数的训练更为高效，从而使模型学习如何快速编辑。ENN 引入了编辑函数和梯度下降编辑器，在内层优化过程中，每次都在一个编辑任务上对模型参数进行较少次数的梯度更新，再用更新后的参数来更新优化器参数。然而，ENN 是针对小型网络 ResNet 设计的，当应用于大型模型时，会面临训练成本高等问题。

为了拓展元学习法在大型模型架构上的应用，KE[2] 将元知识作为超网络，提出了一种通过训练超网络来学习模型参数更新值的方法。在训练超网络时，损失函数由两部分组成，一部分用于确保准确性，另一部分用于确保局部性，并设置了边界值来表示约束的严格程度。训练好的超网络根据输入问题生成模型的参数更新值，使模型能够在特定输入下输出期望的结果，同时保持其他预测不变。为了进一步增强超网络对于大型语言模型的普适性，MEND[19] 通过低秩分解的方法来优化超网络辅助模型参数更新的过程。首先，它利用全连接层中梯度的秩-1 特性，将损失函数关于每一层参数的梯度分解为两个向量的乘积。然后，使用超网络接收分解后的向量作为输入，并输出经过编辑的新向量。最后，这些新向量再次相乘，得到新的参数梯度，并通过一个可学习的缩放因子来计算最终的模型参数更新值。MEND 以较少的参数高效地编辑大型模型，节省了计算资源和内存。

总结来说，基于元学习的编辑方法通过“学习如何编辑”来提高模型在面对新编辑任务时的适应性和泛化能力，能够从一系列编辑任务中提取通用的知识，进而在遇到未见过的编辑任务时，仅使用少量样本训练即可快速收敛，从而节省计算资源和时间。然而，元学习编辑方法也存在不足之处。该方法训练过程较为复杂，应用于大型模型时常常面临训练成本高的问题。尽管 KE 和 MEND 通过使用超网络和梯度低秩分解等技术优化了参数更新过程，减少了计算资源的需求，但仍需进一步提升其对更复杂任务和更大规模模型的适应性与效率。此外，元学习编辑方法从全局视角对模型参数进行更新，即使添加了与局部性相关的损失函数，也有可能对模型原本的知识产生影响，导致模型的不稳定。

2. 定位编辑法

与元学习法相比，定位编辑法修改的是原始模型的局部参数。其先定位到需要修改的参数的位置，然后对该处的参数进行修改。实现定位前需要了解大语言模型中知识的存储机制。当前，对知识存储机制的探索主要依靠定性实验来完成。通

过在一个 16 层 Transformer 的语言模型上进行实验，可以得到结论：**Transformer 中的全连接前馈模块可以看作存储知识的键值存储体 [8]**。其中，全连接前馈模块的输入称为查询（query）向量，代表当前句子的前缀；将全连接前馈模块的上投影矩阵中的向量称为键（key）向量，下投影矩阵中的向量称为值（value）向量。

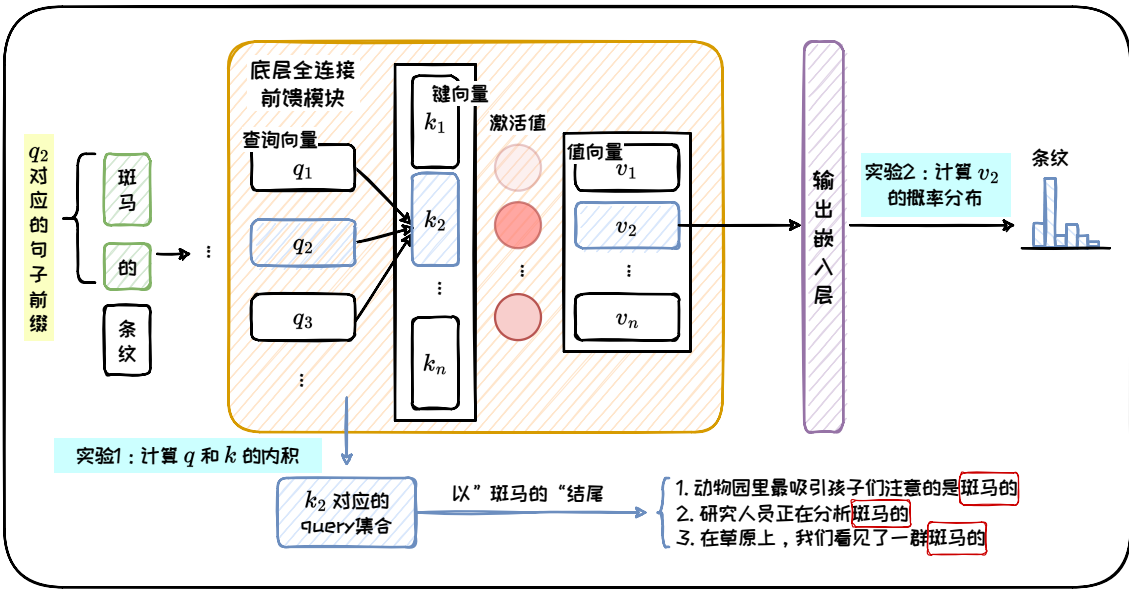


图 5.8: Transformer 可看作键值存储体。

图 5.8展示了文献 [8] 在模型底层全连接前馈模块上进行实验的过程。在实验 1 中，将每个 key 都分别与所有 query($q_1, q_2, q_3, \dots$) 做内积运算，图中以 k_2 为例展示了这一过程。将 k_2 与所有 query 进行内积运算后，发现 k_2 与包括 q_2 在内的几个 query 的激活值较大。将每个 key 对应的激活值较大的 query 收集起来作为一个集合，在该集合中观察到这些 query 对应的前缀都有着相同或相似的模式。例如，图中 k_2 对应的 query 都以” 斑马的 “结尾，也就是说， k_2 存储了与 “斑马的” 相关的文本模式。在实验 2 中，将每个 key 对应的 value 与输出嵌入层（Output Embedding Layer）矩阵相乘，并应用 softmax 函数转换为概率分布，然后比较这些分布与对应 query 的下一个词的相关性，结果发现二者高度相关。例如，图中将 k_2 对应的 v_2 进行上述操作，发现下一个词为” 条纹 “的概率最大，这与 q_2 的下一个词相符。

综合实验 1 和实验 2，文献 [8] 指出：在全连接前馈模块中，模型通过 key 总结了当前 query 所代表的句子前缀的特征，又通过类似键值匹配的机制查找到了相应的 value，也就是下一个词的概率分布。也就是说，Transformer 中的全连接前馈模块可以看作键值存储体对知识进行存储。

基于上述结论，KN[4] 提出了知识神经元的概念。其将全连接前馈模块的每个中间激活值定义为一个知识神经元，并认为知识神经元的激活与相应知识点的表达密切相关。为了评估每个神经元对于特定知识预测的贡献，KN 将有关该知识点的掩码文本输入给预训练模型，获取隐藏状态后，将其输入到模型每一层的 FFN 中，通过归因方法，积累每个神经元在预测正确答案时的梯度变化，从而确定哪些神经元在知识表达过程中起关键作用。这种方法不仅能够识别出显著影响知识表达的神经元，还能过滤掉那些对知识预测贡献较小的神经元。在确定了知识神经元对于知识预测的贡献后，可以通过直接在模型中修改特定知识神经元对应的键向量，来诱导模型输出的编辑后的知识，从而达到模型编辑的效果。

此外，ROME[17] 设计了一种因果跟踪实验，进一步探索中间层全连接前馈模块与知识的关系，优化了知识存储机制的结论。在编辑方法上，与定位和编辑全连接前馈模块中的单个神经元的 KN 不同，ROME 提出更新整个全连接前馈模块来进行编辑。通过 ROME 对 GPT-J 模型上进行编辑，在准确性、泛化性和局部性方面都有良好表现。因此，ROME 成为近年来备受瞩目的模型编辑方法，为未来的模型编辑和优化工作提供了重要的参考和指导。我们将在第 5.4 节详细介绍 ROME 中对于因果跟踪实验和编辑方法的设计。MEMIT[18] 在 ROME 的基础上扩展到对不同知识的大规模编辑，可以同时执行数千次编辑。

总的来说，定位编辑法修改大语言模型的局部参数，在保持模型整体结构和性能的同时，能够对特定知识点进行精准的更新和编辑。与其他方法相比，定位编辑法可同时保持较高的准确性、泛化性和局部性，且适用于各类模型。

5.2.3 方法比较

上述各种模型编辑方法各有优劣。本书参考文献 [27] 中的实验结果, 对主流模型编辑方法在各个性质上的表现进行对比, 结果如表 5.2 所示。表中用“高”、“中”、“低”三个级别定性表示方法的准确性、泛化性、可迁移性和局部性, 用“✓”和“✗”来表示方法的高效性, 即是否支持批量编辑。标注“-”的表明未进行测试。

表 5.2: 模型编辑方法比较。

		方法	准确性	泛化性	可迁移性	局部性	高效性
外部 拓展 法	知识缓存法	SERAC	高	高	低	高	✓
	附加参数法	CaliNET	低	低	-	中	✓
		T-Patcher	高	高	高	中	✗
内部 修改 法	元学习法	KE	低	低	-	高	✓
		MEND	中	高	中	高	✓
	定位编辑法	KN	中	低	-	中	✗
		ROME	高	高	高	高	✗
		MEMIT	高	高	高	高	✓

从表 5.2 中可看出, 在外部拓展法中, 基于知识缓存的 SERAC 在无需额外训练的情况下提供了高效的编辑能力, 保证了高准确性、泛化性和局部性, 适合快速响应和批量编辑, 但可迁移性较差, 编辑缓存和推理模块仍有待优化。基于附加参数的 CaliNET 和 T-Patcher 提供了对模型的直接编辑能力, 但 CaliNET 对不同模型和数据的适应性较差, 而 T-Patcher 虽然保持了高准确性、泛化性和可迁移性, 但在批量编辑时, 对内存的需求较高。

在内部修改法中, 基于元学习法的 KE 和 MEND 将元知识看作超网络, 通过使用模型“学习如何编辑”, 提高了泛化性和训练效率, 且支持批量编辑。然而, 基于元学习的方法训练过程设计较为复杂, 在应用于大型模型时可能受限。基于定位编辑的 KN、ROME 和 MEMIT 则专注于精确定位和编辑模型内部的特定知识。